



Shaheed Zulfikar Ali Bhutto
Institute of Science & Technology

DESIGN AND ANALYSIS OF ALGORITHMS

***PROJECT REPORT: DISASTER RELIEF
MANAGEMENT SYSTEM***

Submitted to: M. Ali Fatmi

Our Team:

Naushaba Asif, Rohban Tariq

Registration ID: 2412485, 2412488

BSCS 4I

1. Problem Statement

During a multi-site humanitarian crisis, disaster relief operations face critical resource constraints. Inefficient distribution leads to a misallocation of vital supplies (food, water, First-aid kits), where low-risk environments might absorb stock while high-risk catastrophic zones are starved of immediate aid.

Additionally, standard systems suffer from a lack of operational checkpoint tracking, making it impossible to safely roll back erroneous or corrupt deployments. To solve this, a system must be built that dynamically analyzes regional severity scores, calculates mathematically bounded allocations under shifting inventory stocks, and manages active regions without relying on slow disk I/O data streams.

2. Objectives

- 1: Implement a custom Max-Priority Queue from scratch to automatically schedule disaster areas based on emergency severity rather than registration time.
- 2: Design a bounded Greedy Resource Allocation Algorithm that scales relief materials dynamically based on region population and injury counts to serve as a mathematically sound recommendation baseline.
- 3: Build a reference-linked History Stack to track deployment states, enabling instant, linear-time chronological rollback (\$Undo\$) checkpoints.
- 4: Maximize application speed and execution stability by running an In-Memory Sandbox Session directly within RAM, completely bypassing slow text-file reading and writing latencies.

3. Implemented Data Structures

A. Custom Linked List Based Max-Priority Queue (PriorityQueue.java)

Unlike normal arrays or Java's built-in collections, this structure manages data using a linked chain of `PriorityNode<T>` elements. Every node contains its data object along with an explicit integer priority score. When a new area is inserted (`enqueuePriority`), the system traverses the list to inject the node into its exact sorted position based on its severity rank.

B. Custom Stack Using Linked List (Stack.java)

This structure processes historical logs using a linear chain of `Node<T>` wrappers under a strict Last-In, First-Out (LIFO) approach. New deployment checkpoints are pushed to the head of the chain in constant time, allowing the system to naturally step backward through history during undo prompts.

4. Implemented Algorithms

A. Greedy Priority Scheduling Algorithm

The system uses a pure greedy strategy for selecting tasks. Instead of iterating through multiple separate lists or keeping track of complex state counters, the scheduler looks directly at the root head node of the PriorityQueue. At any given moment, the region with the highest severity score is processed first.

B. Bounded Greedy Suggestion Framework with Manual Control Override

Located inside DispatchManager.calculateRecommendedSupplies(), this algorithm calculates exact supply requirements against available warehouse limits. Rather than relying on arbitrary metrics, it computes optimal resource distributions by applying explicit logistical demand factors against a dynamic, floating-point Emergency Urgency Ratio:

$$\text{Emergency Urgency Ratio} = \frac{\text{Severity Score}}{10.0}$$

This ratio scales down resource consumption for low-risk zones while unleashing full distribution weight for high-crisis zones. The system evaluates three independent algorithmic equations to generate unique baseline recommendations:

1. Food Boxes Allocation Formula

$$\text{Recommended Food} = \text{Population} \times 0.1 \times \left(\frac{\text{Severity Score}}{10.0} \right)$$

- 0.1 (Food Demand Factor): Models standard emergency response metrics, assuming a critical region requires immediate food packet distribution matching 10% of its total population footprint in the initial wave.

2. First Aid Kits Allocation Formula

$$\text{Recommended Medical} = \text{Injuries} \times 2.0 \times \left(\frac{\text{Severity Score}}{10.0} \right)$$

- 2.0 (Medical Risk Multiplier): Assumes that for every single reported injury, the system must allocate 2 complete First Aid kits. This scaling accounts for supply contamination risk, procedural triage depth, and crucial secondary field backups.

3. Water Bottles Allocation Formula

$$\text{Recommended Water} = \text{Population} \times 0.3 \times \left(\frac{\text{Severity Score}}{10.0} \right)$$

- 0.3 (Hydration Hydrology Scalar): Mandates that water hydration blocks are allocated at 30% of the total population metric, mathematically acknowledging that hydration volumes represent a significantly higher physical necessity than solid food parcels during an active crisis.

4. The Upper Boundary Ceiling Clamp

To guarantee system stability, the resulting calculations are passed through a data-layer safety gate before rendering on screen or accepting manual overrides:

$$\text{Final Allocated Amount} = \min(\text{Recommended Quantity}, \text{Available Warehouse Stock})$$

If an entered value exceeds warehouse availability, an execution exception is thrown, halting the transaction and leaving the queue completely unchanged.

5. System Design

The application uses a clean, decoupled architecture:

- **AreaManager:** Interacts with the custom PriorityQueue to handle adding and peeking at affected regions.
- **DispatchManager:** Houses the greedy calculation engine and updates the history Stack timeline.
- **InventoryManager:** Validates and updates active supply balances in-memory.

6. Time Complexity Analysis

A. Priority Queue Insertion (addArea)

- **Worst-Case:** $O(N)$, where N is the number of regions currently queued. If a newly registered area has the lowest severity score (1), the traversal pointer has to walk sequentially from the head node to the very end of the list before finding its correct place.
- **Best-Case:** $O(1)$ constant time. If an incoming area has a peak severity score (10), it immediately updates the head node reference without traversing the list.

B. Priority Queue Extraction (serveNextArea)

- **Worst/Best Case:** $O(1)$ constant time. Because the underlying custom structure pre-sorts items during insertion, fetching and dequeuing the top priority target requires no scanning or list iteration.

C. Greedy Supply Computation (calculateRecommendedSupplies)

- **Worst/Best Case:** $O(1)$ constant time. The formula evaluates raw field values using basic mathematical equations and executes boundary checks without any loops.

D. Undo Functionality (undoLastDispatch)

- **Worst-Case:** $O(B)$, where B is the number of item records attached to the most recent deployment batch ID. The history tracking stack pops records matching the batch ID one by one to accurately restore warehouse balances.

7. Space Complexity Analysis

- **System State Queue:** $O(N)$ auxiliary space, where N represents the maximum number of active disaster locations stored in RAM at the same time.
- **Timeline History Stack:** $O(H)$ auxiliary space, where H is the total number of resource allocation records processed within a single session.
- **Algorithmic Overhead:** $O(1)$ auxiliary space. The greedy formula uses a fixed array footprint size of 3 items (`int[3]`), introducing zero runtime memory growth.

8. Results and Testing

Testing was conducted within an isolated sandbox environment preloaded with baseline evaluation datasets (Lahore [Sev: 8, Pop: 90,000, Injuries: 900], Korangi [Sev: 7], Larkana [Sev: 4], Landhi [Sev: 2]).

- **Test Case 1: Priority Order Verification.** Executing Menu Option 6 displayed the pending priority list. Lahore was automatically placed at position 1 due to its high severity score (8), validating the insertion sorting mechanics of our custom PriorityQueue.
- **Test Case 2: Greedy Recommendation and Manual Control Gate.** Selecting Menu Option 2 successfully generated mathematically ideal suggestions for Lahore (Food Boxes: 7,200, First Aid Kits: 1,440, Water Bottles: 21,600). To test our system security features, an intentional manual error entry of 600000 food boxes was supplied. The inventory layer instantly threw an exception due to insufficient warehouse stock, safely halting the transaction and keeping Lahore securely at the top of the queue. Re-running the case with the validated suggested values processed cleanly.
- **Test Case 3: Sequential Checkpoint Rollback.** Selecting Menu Option 3 popped the history stack and the system returns the exact user-defined custom amounts back to the inventory balances and restores Lahore back to position 1 at the head of the priority queue.

9. Conclusion

This project successfully demonstrates an algorithmic framework for resource distribution under strict operational constraints, built entirely from scratch without external third-party collections libraries. By utilizing an in-memory sandbox architecture rather than disk-bound file operations, the system isolates algorithmic performance from hardware latencies and ensures execution stability. The combination of a custom Max-Priority Queue for greedy target selection, a bounded multi-variable calculation framework for allocation recommendations, and a reference-linked stack for linear-time transaction rollbacks provides a robust, theoretically sound solution for real-time crisis coordination.

10. References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
2. Sahni, S. (2005). Data Structures, Algorithms, and Applications in Java (2nd ed.). Silicon Press.